

Tor: Onion Routing and Bidirectional Anonymity

NECKER

November 18, 2025

Contents

1	Introduction to Tor (The Onion Router)	2
1.1	Principles of Onion Routing	2
2	Comparison of Security, Privacy, and Anonymity Levels	3
2.1	1. Unencrypted Connections (Normal HTTP)	3
2.2	2. Encrypted Connections (Normal HTTPS)	3
2.3	3. Anonymity with Unencrypted Traffic (Tor without HTTPS)	3
2.4	4. Anonymity and Encryption (Tor with HTTPS)	4
3	Building a Tor Circuit	5
3.1	Creation Phases	5
3.2	Communication Phase	5
4	Bidirectional Anonymity: Hidden Services	6
4.1	Structure and Roles of Specialized Nodes	6
4.2	Connection Establishment Process	6
4.3	Onion Service Connection Establishment Process	6
4.4	Rendezvous: The Final Meeting	7
5	Limitations and Drawbacks of Tor	8

1 Introduction to Tor (The Onion Router)

Tor is a system that aims to guarantee anonymity in Internet communication by routing traffic through a global voluntary network of relays (nodes). Anonymity is achieved through the technique of **Onion Routing**.

1.1 Principles of Onion Routing

Information is encapsulated in successive layers of encryption, like the layers of an onion. Each relay (node) along the path removes a layer of encryption, revealing only the address of the next relay.

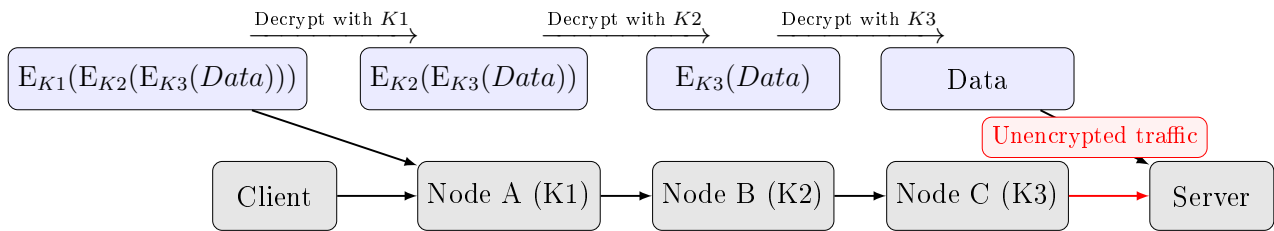


Figure 1: Principle of Onion Routing (Tor).

The diagram (Fig. 1) illustrates how the message is encrypted with the public keys of all nodes in reverse order ($K3, K2, K1$) and decrypted sequentially by each node:

- **Node A** (Guard) removes $K1$, knowing only the Client's identity and the address of Node B.
- **Node B** (Middle Relay) removes $K2$, knowing only the address of Node A and the address of Node C.
- **Node C** (Exit) removes $K3$, knowing the address of Node B and the **destination** (the Server). It does not know the identity of the original Client.

2 Comparison of Security, Privacy, and Anonymity Levels

The protection of a user's data and identity critically depends on the combination of protocols used. Below, we analyze the different connection scenarios.

2.1 1. Unencrypted Connections (Normal HTTP)

This is the least secure scenario, typical of old HTTP connections not protected by SSL/TLS.

- **Encryption: Absent.**
- **Traffic Visibility:**
 - **Anyone can see everything:** Data (passwords, messages), the Client's IP address, and the content of the request are transmitted in **cleartext**. Any entity monitoring the network (Internet service providers, routers, attackers) can read the information.
- **Privacy and Security:** Extremely low. Unsuitable for transmitting sensitive information.

2.2 2. Encrypted Connections (Normal HTTPS)

This is the minimum security standard for the modern web, where the HTTPS protocol utilizes TLS/SSL encryption.

- **Encryption: End-to-End Encryption** between the Client and the website Host (using the site's private and public keys to establish a symmetric key).
- **Traffic Visibility:**
 - **Hidden Content:** Data (passwords, message bodies) are encrypted and visible only to the Host.
 - **Visible Metadata:** External entities (ISPs, attackers) can see the Client's IP address and the Server's IP address (the website being visited).
- **Privacy and Security:** High security for content, but **the Client's identity and location are not anonymous**. The website Host sees everything (IP, decrypted data) and can share this information.

2.3 3. Anonymity with Unencrypted Traffic (Tor without HTTPS)

A scenario where the Tor network is used for anonymity, but the accessed website uses only HTTP.

- **Encryption: Layered encryption** within the Tor network (between the Client and adjacent Relays). **Absent** between the Last Relay and the destination Server.
- **Traffic Visibility:**
 - **Inside Tor:** The connection is secure thanks to onion encryption.
 - **At the Exit (Exit Node):** After the last Relay (Exit Node), data travels in **cleartext** toward the destination Server. The Exit Node can therefore see the entire content of the communication (if it is not HTTPS).
- **Anonymity:**

- **Location:** The real location of the Client is known only to the **First Relay** (Guard Node).
- **Vector Attack:** Anonymity is at risk. The **Exit Node** can spy on the entire content of the communication. **Intermediate Relays** cannot spy on the destination website.
- **Conclusion:** Not recommended. Anonymity is guaranteed, but not the confidentiality of the content.

2.4 4. Anonymity and Encryption (Tor with HTTPS)

This scenario offers the highest level of protection, combining Tor's anonymity with the end-to-end encryption of HTTPS.

- **Encryption:**
 - **Tor:** Onion encryption between the Client and the Exit Node.
 - **HTTPS:** End-to-End encryption between the **Exit Node** and the **Website Host**.
- **Traffic Visibility:**
 - **Exit Node:** The exit Relay **cannot decrypt** the content of the communication, as it is encrypted via HTTPS (visible only to the website Host). It only sees that it is forwarding encrypted traffic to a specific IP address.
 - **Website Host:** The Host sees all the content of the communication (once decrypted) and sees the IP address of the **Exit Node**.
- **Anonymity:**
 - **Location:** The real location of the Client is known only to the First Relay.
 - **Confidentiality:** Data is visible **only** to the Host (and any third parties with whom the Host shares the data).
- **Conclusion:** Maximum security. It offers both anonymity (hides the real IP) and confidentiality (hides the content from all Relays and interceptors).

Summary of Risks

- The level of **Anonymity** is primarily determined by **Tor**.
- The level of **Content Confidentiality** is primarily determined by **HTTPS**.

3 Building a Tor Circuit

A Tor circuit is a chain of N relays (generally three, R_1, R_2, R_3) established between the client and the destination server.

3.1 Creation Phases

1. **Relay Selection:** The client chooses a chain of N routers ($R_1 \dots R_n$) from a known list (Directory Nodes).
2. **Initial Asymmetric Encryption:** The client knows the **public keys** of all selected relays.
3. **Onion Creation:** The message to be sent to the Server is encrypted in layers, one for each relay.
4. **Circuit Extension (Handshake):** The handshake phase does not involve a single message but rather one for each router, building the path step by step (more secure than a single message):
 - The client sends a **CREATE** message to the first router (R_1) encrypted with R_1 's public key. This establishes a temporary symmetric key (K_1) (the client requests it and both establish it, so there is no need to send it back; a crucial point to understand, otherwise when R_2 sends its key back, R_1 would know it).
 - To extend the circuit to R_2 , the client creates a new message:
 $E_{K_1}(\text{Extend to } R_2, E_{K_{\text{pub}, R_2}}(\text{Key}))$.
 - R_1 decrypts it with K_1 , sees that it must extend the circuit to R_2 . It forwards the message to R_2 , which establishes the symmetric key K_2 with the *Client*.
 - The process repeats for all nodes, establishing a unique symmetric key (K_m) between the Client and each node R_m .

3.2 Communication Phase

- Once the connection is established, the client will encrypt the message with all symmetric keys.
- The server will simply send the response to the last node, which will encrypt it with its symmetric key and send it to the previous node, and so on until it reaches the client.

Table 1: Anonymity Roles in a Tor Circuit with $N = 3$

Node	Knows Client's Identity?	Knows Destination?
R_1 (Guard)	Yes (Direct communication)	No (Knows only R_2)
R_2 (Middle)	No (Knows only R_1 and R_3)	No (Knows only R_1 and R_3)
R_n (Exit/Last Relay)	No (Knows only R_{n-1})	Yes (Communicates with Server)

4 Bidirectional Anonymity: Hidden Services

Onion Services (Hidden Services) guarantee bidirectional anonymity, hiding both the location of the Client and that of the Server (Host). Communication is established through a secret meeting point called the Rendezvous Point (RP).

4.1 Structure and Roles of Specialized Nodes

- **Directory Nodes (HSDir):** (A sort of **DNS**) They function as a public and distributed registry where Hosts publish their descriptors (IPs of the **Introduction Points**, their public **keys**, and the host's **signature** to verify it is truly them).
- **Introduction Point (IP):** Tor nodes chosen by the Host to receive connection requests (the end-point of the onion connection, just like for the client).
- **Rendezvous Point (RP):** A Tor node chosen by the Client that acts as the final bridge for the anonymous meeting (the endpoint of the first Tor circuit established by the client).

4.2 Connection Establishment Process

The process is divided into two parallel flows (Host and Client) that converge at the Rendezvous Point.

4.3 Onion Service Connection Establishment Process

The process is structured into two distinct but parallel flows (Host and Client) that converge at the Rendezvous Point (RP) to guarantee bidirectional anonymity.

- **A. Hidden Service Process (Host)**

The Host takes the following steps to make itself anonymously reachable:

1. **Selection of Introduction Points (IP):** The Host (Server) randomly chooses N Tor nodes from the Relay list to act as Introduction Points (**IP**).
2. **Building Long-Lived Circuits:** The Host establishes a **long-lived anonymous Tor circuit** to each IP. The IP acts as the last node of this circuit and remains blind to the real IP of the Host.

$$\text{Host} \xrightarrow{\text{Anonymous Circuit}} \text{IP}_m$$

3. **Publishing the Descriptor:** The Host creates a **Descriptor** (a metadata document) containing:
 - The **public key** of the Host (derived from the `.onion` address).
 - The **addresses** of all chosen IPs.

The Descriptor is **cryptographically signed** by the Host (with its private key) to ensure its authenticity and integrity. It is then published to the **HSDir** (Directory Nodes dedicated to Hidden Services).

- **B. Client Process**

The Client takes the following steps to locate and initiate the meeting with the Host:

1. **Retrieving the Descriptor:** The Client, knowing the `.onion` address, queries an **HSDir** to retrieve the Descriptor published by the Host. The Client **verifies the signature** to authenticate the Descriptor.
2. **Choosing and Building the RP Circuit:** The Client randomly chooses a Tor node to act as a **Rendezvous Point (RP)** and establishes its own **anonymous Tor circuit** toward it.

Client $\xrightarrow{\text{Anonymous Circuit}}$ RP

3. **Introduction Request:** The Client selects one of the IPs from the Descriptor and sends an **Introduction Request** through its standard Tor circuit toward the IP. The request contains two fundamental pieces of information:

- The address of the **RP** (where the meeting will take place).
- A **One-Time Secret** code that will be used to verify identity at the RP.

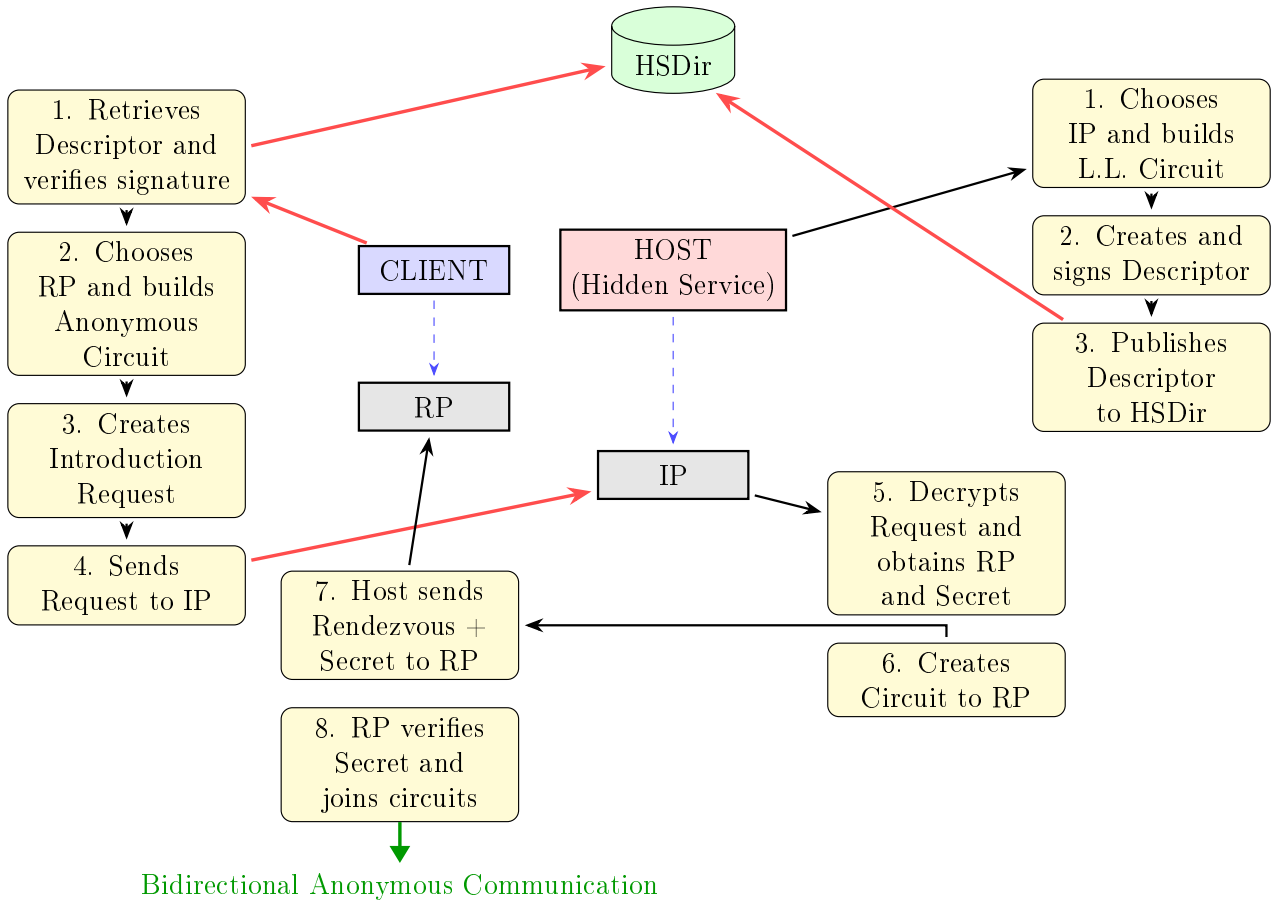
The Payload (RP and Secret) is **encrypted with the Host's public key** so that only the Host can read it.

4.4 Rendezvous: The Final Meeting

4. **Host Connection to the RP:** The IP forwards the Introduction request (along the long-lived circuit) to the Host. The Host decrypts the message, extracts the RP address and the One-Time Secret. Subsequently, the Host establishes a **new anonymous Tor circuit** that ends at the RP.

Host $\xrightarrow{\text{Anonymous Circuit}}$ RP

5. **Joining the Circuits:** The Host sends a Rendezvous message to the RP (including the One-Time Secret). The RP verifies the secret code with the one provided by the Client. Once confirmed, the RP joins the two anonymous circuits.
6. **Bidirectional Communication:** The connection is established. Traffic flows between Client and Host (in both directions) through the RP, guaranteeing complete anonymity to both parties.



Step Diagram for Connecting to Tor Hidden Services

5 Limitations and Drawbacks of Tor

Although Tor offers strong anonymity, it presents some critical limitations:

- **Latency and Speed:** Routing traffic through N relays (each with encryption/decryption operations) and the voluntary nature of nodes cause a **significant reduction in speed** and an increase in latency.
- **Bandwidth:** Each Tor node serves multiple users simultaneously. The nodes, especially those managed by private users, require **high bandwidth** and can leave the circuit at any moment (if they turn off their PCs, all connections are interrupted).
- **Vulnerability at the Exit (Exit Node):** Although traffic within the circuit is encrypted, the traffic between the Exit Node (R_n) and the destination Server is **NOT encrypted** if the destination site does not use HTTPS. The exit node can intercept data in cleartext.
- **De-Anonymization Attacks:** Tor is not immune to attacks aiming to breach anonymity:
 - **Traffic Correlation Attacks:** If an attacker controls both the Guard node (R_1) and the Exit node (R_n), they can correlate the entry and exit times of packets to identify the Client (End-to-End Attack).
 - **Sybil Attacks:** An attacker injects many malicious nodes into the Tor network to increase the probability of controlling the entire circuit.

- **Software Requirements:** Utilizing the Tor browser often requires a customized edition of Firefox to prevent *fingerprinting* (servers can log client hardware details or time zones, etc., to later match this info and recognize you even if you changed your IP) and other browser-based vulnerabilities.